



DOCUMENT PARTAGE SAE 4.01 ÉQUIPE NEPTUNE 2

Projet : Diffusion Sonore

[Dépôt](#)

Analyse de l'existant

Equipe	Analyse
Neptune A	Code : - Le site est bien avancé, l'architecture MVC est bien définie et respectée dans le code. - Il manque cependant plusieurs choses comme la finalisation du dashboard du rôle commercial ou encore la possibilité de savoir exactement quelle musique joue sur le lecteur depuis le site. - Base de Données : - Pas de boucle - La base de données est centralisé sur les concepts des utilisateurs et des entreprises, mais celle-ci peut vite rendre flou son exploitation dans le code ensuite - le principe d'ip pour les lecteurs est bien mais en pratique il faut s'assurer que le réseau ne change pas du côté de celui-ci et que celle-ci soit constamment la même. Ce principe ne marche théoriquement ici que pour un réseau local.
Neptune B	Code : - Points forts : - Architecture MVC/DAO plutôt bien respectée dans l'ensemble, la séparation des responsabilités est bien faite - Il y a la présence d'un service de traduction - Points faibles : - les noms des fichiers/variables/fonctions ne sont pas calqués sur une convention précise ce qui ne facilite pas la lecture du code et sa modification - l'initialisation de la base de données qui est décentralisé ce qui est problématique Base de Données : - Difficulté de compréhension de l'utilité de la table Serveur et son lien avec les entreprises, cela rend la base de données difficile à cerner et à exploiter. - La table Playlist a idMusique en clé étrangère qui fait référence à la table Musique, c'est difficile de comprendre pourquoi une playlist n'aurait qu'une seule musique ?
Neptune C	Code : - Architecture MVC respectée et bien définie dans le code. - Bonne compréhension du code de l'application permettant de savoir exactement où se trouve quelconque problème. - Base de Données : - Plusieurs cardinalités sont inversées notamment pour Rôle(1-1) et Utilisateur (0,n). Ça devrait être l'inverse de cette cardinalité. C'est exactement pareil pour Type_Contentu (1,1) et Fichier_audio (0,n). - Redondance dans la base de données avec Date_ajout qui se retrouve dans la table Fichier_audio et la table Ajoute

Neptune E	Code : - Architecture MVC plutôt respectée (sur la logique) - Absence des DAO, Controller, DAOInterface et de Service pour Playlist, présence unique de 'Playlist.py'. Cela veut dire que la logique liée aux playlists est totalement éparpillée dans un autre contrôleur à part et que ce même contrôleur se retrouve à gérer des responsabilités qui n'est pas la sienne. De plus, sans aucun DAO ni Service, il est impossible de tester différentes règles lié à une Playlist. Base de Données : - Il n'y a pas de schéma SQL ni de base de données disponible sur leur dépôt. Le fichier 'final_schema.sql' est vide et il ne génère également pas de fichier

Choix du projet de S3 ; Synthèse

À partir de l'analyse des projets de S3, nous avons décidé de continuer sur le même projet que le semestre précédent, c'est-à-dire continuer de développer sur le projet du groupe Neptune D. L'analyse de l'existant des projets du semestre 3 (S3) des autres équipes a permis de démontrer plusieurs lacunes techniques sur la structure du code et les bases de données. C'est à partir de ces analyses que nous avons décidé de continuer sur le même projet que le semestre précédent, c'est-à-dire continuer de développer sur le projet du groupe Neptune D.

Ce choix se justifie par l'Absence des DAO, Controller, DAOInterface et de Service pour Playlist, présence unique de 'Playlist.py' notamment chez le groupe Neptune E ce qui rend impossible à tester, le manque de conventions des fichiers/variables/fonction non calqués qui ne facilite pas la lecture du code et sa modification, l'inachèvement de leur dashboard et le retour d'information en temps réel sur le lecteur de musique manquant chez Neptune A. De plus, pouvoir garder notre base de donnée nous évite de devoir corriger de lourds problèmes lors de sa création des autres groupes de projet (cardinalités, absence complète de la base de donnée...) ou de reconstruire entièrement une nouvelle base de données..

Pour conclure, le choix de poursuivre sur le projet Neptune D est un choix d'efficacité et de qualité. Plutôt que de subir les empêchements technique des autres groupes, ce choix vous permet de capitaliser sur une architecture MVC saine et maîtrisée et une base de données en accord avec nos objectifs tout en gardant une base solide et simple à améliorer..

Analyse des Besoins

Diagramme de cas d'utilisation :



Dans le projet réalisé en S3, on a préféré exprimer les besoins en utilisant des user stories. Ces user stories présentaient plusieurs acteurs comme l'Opérateur de diffusion, le Superviseur technique, le Lecteur de diffusion, l'Équipe de développement et le Client. Cependant, cette première analyse restait assez générale et ne représentait pas vraiment les rôles utilisés dans l'application finale.

L'ancienne gestion des rôles peut être critiquée. Par exemple, l'Opérateur de diffusion regroupait trop de responsabilités différentes comme la gestion des playlists, la consultation de l'état des lecteurs, le déclenchement de messages urgents et la consultation de l'historique. Dans la nouvelle version du projet, on a séparé ces responsabilités en deux rôles : Marketing et Commercial. Le rôle Marketing est chargé de créer, de modifier, de planifier et de synchroniser les playlists, tandis que le rôle Commercial s'occupe surtout des publicités et des annonces.

Le rôle de Superviseur technique correspond surtout au rôle Marketing dans l'application actuelle. Ce rôle supervise la programmation musicale : il peut créer une playlist, modifier une playlist, planifier une playlist, gérer les fichiers audios, synchroniser les playlists et consulter l'historique de diffusion. Le terme "Superviseur technique" utilisé dans les user stories n'était donc pas le bon terme, car dans le site actuel cette responsabilité est surtout liée à la supervision des playlists et de la diffusion musicale.

Le rôle Admin / Technicien a été ajouté dans le nouveau diagramme, même s'il a été utilisé pour le projet du s3, il n'était pas présent dans les user stories. Il peut gérer les utilisateurs et les rôles, gérer les organisations, gérer les lecteurs, consulter le tableau de bord, consulter l'état des lecteurs, consulter les alertes et tester la continuité de service.

L'acteur Client présent dans l'analyse initiale n'a pas été conservé comme acteur direct dans le nouveau diagramme. En effet, le produit est destiné à des organisations clientes, mais ces organisations utilisent l'application à travers différents profils utilisateurs : Admin / Technicien, Marketing et Commercial.

On a aussi ajouté des précisions concernant le Lecteur de diffusion. Dans le nouveau diagramme, il est représenté par le Lecteur Raspberry. Ce n'est pas un utilisateur humain, mais un acteur technique qui communique avec le serveur. Il peut envoyer son état au serveur, envoyer le contenu en cours de diffusion, diffuser de la musique, des publicités ou des annonces, et basculer sur une playlist de secours.

Le nouveau diagramme contient aussi l'analyse avec une fonctionnalité qui n'était pas présente dans le projet de S3 : le choix d'une ambiance musicale. Cette fonctionnalité permet de personnaliser la diffusion selon le besoin de l'organisation, dès la connexion. Par exemple avec une ambiance joyeuse, calme, romantique ou dynamique. Elle améliore l'expérience utilisateur en adaptant le contenu musical et l'identité visuelle du site à l'ambiance choisie. L'ambiance peut être changée à tout moment.

Le nouveau diagramme de cas d'utilisation est donc plus complet que l'analyse initiale. Il conserve les besoins principaux du projet de S3, comme la gestion des playlists, la diffusion d'annonces, la supervision des lecteurs et l'historique de diffusion, mais il les réorganise selon les rôles réellement utilisés dans l'application. Il ajoute également des fonctionnalités manquantes du point de vue utilisateur, notamment la personnalisation de l'ambiance musicale.

lien de la planification: [📅 Planification SAE](#)